



CMPT 413/713: Natural Language Processing

Language Models

Fall 2021
2021-09-14

Adapted from slides from Anoop Sarkar, Danqi Chen and Karthik Narasimhan

Announcements

- Office hours start this week
 - Sonia: Mon 1:30-2:30pm (zoom)
 - Jetic: Wed 11:00-12:00pm (TASC 9404 or zoom)
 - Ali: Tue 9:30-10:30am (google duo)
(this week Fri 9:30-10:30am)
- My office hours: 1-2pm Tues (sign up for 10min slots)
- Optional tutorial on getting started with python

Announcements

- Additional video lectures (optional)
 - (40m) Levels of representation
 - (8m) Traditional NLP pipeline + NLP systems
- Cover terminology and information that is useful

Announcements

- HW0 is due **Thursday 9/16, by 11:59pm!**
- Form your groups! Find some friends!
- ~20 groups already, ~80 students not yet part of a group!
- HW1 will be out Thursday and will be due on **Tuesday 9/28**.
To avoid homework due on holidays
- Note that for Programming Assignments (HW#-P) there will be default of 2 grace days
- However, you **must have submitted something by the initial due date**
- This does not apply for the HW#-C component

What will this course cover?

Representations

- Embeddings
- Compositional
- Structured

Methods

- Rule based
- Statistical Methods
- Neural Models
- Dynamic programming
- Language Models

First two months of the course

Applications and Tasks

- Machine Translation
- Dialogue Agents
- Information Extraction
- Question Answering
- Grounding

Last month of the course

Consider

Today, in Vancouver, it is 76 F and red

vs

Today, in Vancouver, it is 76 F and sunny

- Both are grammatical
- But which is more likely?

Language Modeling

- We want to be able to estimate the probability of a sequence of words
- How likely is a given phrase / sentence / paragraph / document?

Why is this useful?

Applications

- Predicting words is important in many situations
- Machine translation

$$P(\text{a smooth finish}) > P(\text{a flat finish})$$

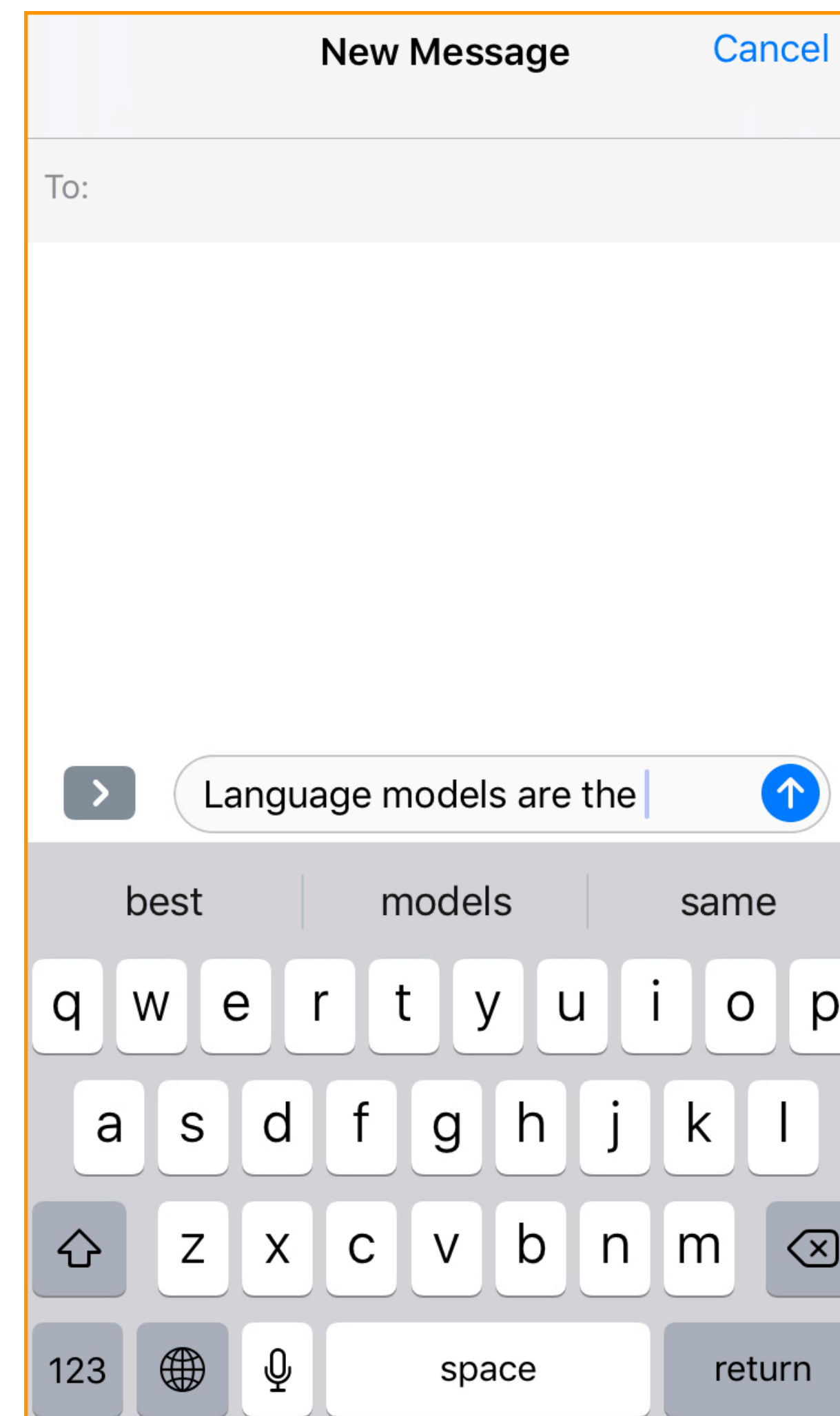
- Speech recognition/Spell checking

$$P(\text{high school principal}) > P(\text{high school principle})$$

- Information extraction, Question answering

Language models are everywhere

Autocomplete



Impact on downstream applications

Language Resources	Adaptation	Word		PP
		Cor.	Acc.	
1. Doc-A		54.5%	45.1%	49972
2. Trans-C(L)		63.3%	50.6%	1856.5
3. Trans-B(L)		70.2%	60.3%	318.4
4. Trans-A(S)		70.4%	59.3%	442.3
5. Trans-B(L)+Trans-A(S)	CM	72.6%	63.9%	225.1
6. Trans-B(L)+Doc-A	KW	72.1%	64.2%	247.5
7. Trans-B(L)+Doc-A	KP	73.1%	65.6%	259.7
8. Trans-A(L)		75.2%	67.3%	148.6

(Miki et al., 2006)

New Approach to Language Modeling Reduces Speech Recognition Errors by Up to 15%



December 13, 2018

Ankur Gandhe

Alexa

Alexa research

Alexa science

What we will cover today

- What is a language model?
- Statistical language model using **ngrams**
- How to build a language model? **Training the model from data**
Learning/estimating model parameters
- MLE and smoothing
- How to use the language model? **Generation**
- How to tell if our language model is working well? **Evaluation**

What is a language model?

Probabilistic model of a sequence of words

Setup: Assume a finite vocabulary of words V

$$V = \{\text{cat, clown, crazy, killer, mat, on, sat, the}\}$$

V can be used to construct an infinite set of sentences (sequences of words)

$$V^+ = \{\text{clown, cat sat, killer clown, crazy clown, crazy cat, crazy killer clown, killer crazy clown, ...}\}$$

where a sentence is defined as $s \in V^+$ where $s = \{w_1, \dots, w_n\}$



What is a language model?

Probabilistic model of a sequence of words

Given a **training data** set of example sentences

$$S = \{s_1, s_2, \dots, s_m\}, s_i \in V^+$$

Estimate a probability model

$$\sum_{s_j \in V^+} P(s_j) = \sum_j P(w_1, \dots, w_{n_j}) = 1.0$$

Language Model

- $p(\text{clown}) = 1\text{e-}5$
- $p(\text{killer}) = 1\text{e-}6$
- $p(\text{killer clown}) = 1\text{e-}12$
- $p(\text{crazy killer clown}) = 1\text{e-}21$
- $p(\text{crazy killer clown killer}) = 1\text{e-}110$
- $p(\text{crazy slow killer killer}) = 1\text{e-}127$
- ...

Where do we get the vocabulary?

Common Setup: Assume a finite vocabulary of words V

- Get from a list of words (say a dictionary)
- Build from training data
- Decide on vocabulary size (say $|V| = 50K$) and then pick most frequent words
- Take words that occur more than T times.



Learning language models

How to estimate the probability of a sentence?

- We can **directly count** using a training data set of sentences

- $$P(w_1, \dots, w_n) = \frac{C(w_1, \dots, w_n)}{N}$$

- C is a function that counts how many times each sentence occurs
- N is the sum over all possible $C(\cdot)$ values

Learning language models

How to estimate the probability of a sentence?

$$P(w_1, \dots, w_n) = \frac{c(w_1, \dots, w_n)}{N}$$

- **Problem:** does not generalize to new sentences unseen in the training data
- What are the chances you will see a sentence?
crazy killer clown crazy killer
- In NLP applications, we often need to assign non-zero probability to previously unseen sentences

Estimating joint probabilities with the chain rule

$$\begin{aligned} P(w_1, w_2, \dots, w_n) &= P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \times \dots \times P(w_n|w_1, w_2, \dots, w_{n-1}) \\ &= \prod_{i=1}^n P(w_i|w_1, \dots, w_{i-1}) \end{aligned}$$

Example

Sentence: “the cat sat on the mat”

$$\begin{aligned} P(\text{the cat sat on the mat}) &= P(\text{the}) \times P(\text{cat}|\text{the}) \times P(\text{sat}|\text{the cat}) \\ &\quad \times P(\text{on}|\text{the cat sat}) \times P(\text{the}|\text{the cat sat on}) \\ &\quad \times P(\text{mat}|\text{the cat sat on the}) \end{aligned}$$

Estimating probabilities

Let's count again!

$$P(\text{sat}|\text{the cat}) = \frac{\text{count}(\text{the cat sat})}{\text{count}(\text{the cat})}$$

$$P(\text{on}|\text{the cat sat}) = \frac{\text{count}(\text{the cat sat on})}{\text{count}(\text{the cat sat})}$$

⋮

Maximum likelihood estimate (MLE)

- With a vocabulary of size $|V|$
- # sequences of length n : $|V|^n$
- Typical vocabulary $\sim 50\text{k}$ words
- even sentences of length ≤ 11 results in $\approx 4.9 \times 10^{51}$ sequences!
(# of atoms in the earth $\approx 10^{50}$)

Way too many parameters!

Markov assumption

- Use only the **recent past** to predict the next word
- Reduces the number of estimated parameters in exchange for modeling capacity

- 0th order

$$P(\text{mat}|\text{the cat sat on the}) \approx P(\text{mat})$$

- 1st order

$$P(\text{mat}|\text{the cat sat on the}) \approx P(\text{mat}|\text{the})$$

- 2nd order

$$P(\text{mat}|\text{the cat sat on the}) \approx P(\text{mat}|\text{on the})$$

k^{th} order Markov

- Consider only the last k words for context

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-k} \dots w_{i-1})$$

which implies the probability of a sequence is:

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i | w_{i-k} \dots w_{i-1})$$

($k+1$) gram

n-gram models

Unigram $P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i)$

Bigram $P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i | w_{i-1})$

Trigram $P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i | w_{i-2}, w_{i-1})$

and 4-gram, 5-gram, and so on.

*Larger the n , more accurate and better the language model
(but also higher costs)*

Caveat: Assuming infinite data!

Estimating n-gram probabilities

- **Maximum likelihood estimate (MLE):** Use counts from text corpus

Unigram
$$P(w_i) = \frac{C(w_i)}{\sum_{w_i \in V} C(w_i)} = \frac{C(w_i)}{N}$$

Bigram
$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Trigram
$$P(w_i | w_{i-1}, w_{i-2}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}$$

Can reuse counts for multiple estimations

Maximum Likelihood Estimation

We want to find the set of parameters $\hat{\theta}$ that maximize the probability of the training data

$$\hat{\theta} = \arg \max_{\theta \in \Theta} \hat{L}(\theta; \mathcal{D})$$

Parameters

$$\theta : \{p(w_i | w_1, \dots, w_{i-1})\}$$

Corpus of N sentences

Using our model, we can estimate the probabilities of these sentences

Likelihood

$$\hat{L}_m(\theta; \mathcal{D}) = \prod_{j=1}^m P(w_1^{(j)}, \dots, w_{n_j}^{(j)}) = \prod_{j=1}^m \prod_{i=1}^{n_j} P(w_i^{(j)} | w_1^{(j)}, \dots, w_{n_j}^{(j)})$$

Log-Likelihood

$$\hat{\ell}_m(\theta; \mathcal{D}) = \sum_{j=1}^m \log P(w_1^{(j)}, \dots, w_{n_j}^{(j)}) = \sum_{j=1}^m \sum_{i=1}^{n_j} \log P(w_i^{(j)} | w_1^{(j)}, \dots, w_{n_j}^{(j)})$$

Easier to work with (products to sums)
Numeric underflow less of a issue

Likelihood function

- How likely it is to see the examples in the training data
- Function of the parameters you are using to model the probability
- Probability density over data samples (sentences)

Typical assumptions

- Samples are iid (independently and identically distributed)

Maximum Likelihood Estimation (for categorical distributions)

- Unigram: $P(w)$
- Probability: $P(w) \geq 0, \sum_{w \in V} P(w) = 1$
- MLE is the **sample mean**
- Optimize $P_{\text{ML}}(w) = \arg \max_{P(w)} \sum_{j=1}^N \sum_{i=1}^{n_j} \log P(w_i^{(j)}) = \arg \max_{P(w)} \sum_{w \in V} C(w) \log P(w)$
- Solve using Lagrange Multipliers $g(\lambda, P(\cdot)) = \sum_{w \in V} C(w) \log P(w) - \lambda (\sum_{w \in V} P(w) - 1)$

$$P(w) = \frac{C(w)}{\lambda} \quad \longrightarrow \quad P(w) = \frac{C(w)}{\sum_{w \in V} C(w)} = \frac{C(w)}{N}$$